

PICCOLO CAFE

Start here. Run the project. Learn how it works.

A guide for your first full stack project
Windows setup, explained step by step



No previous full stack experience required. Learn what the frontend, backend and database do, start the local application, create an account, and follow a request through the code.

Original production data and live deployment remain unchanged.

Contents

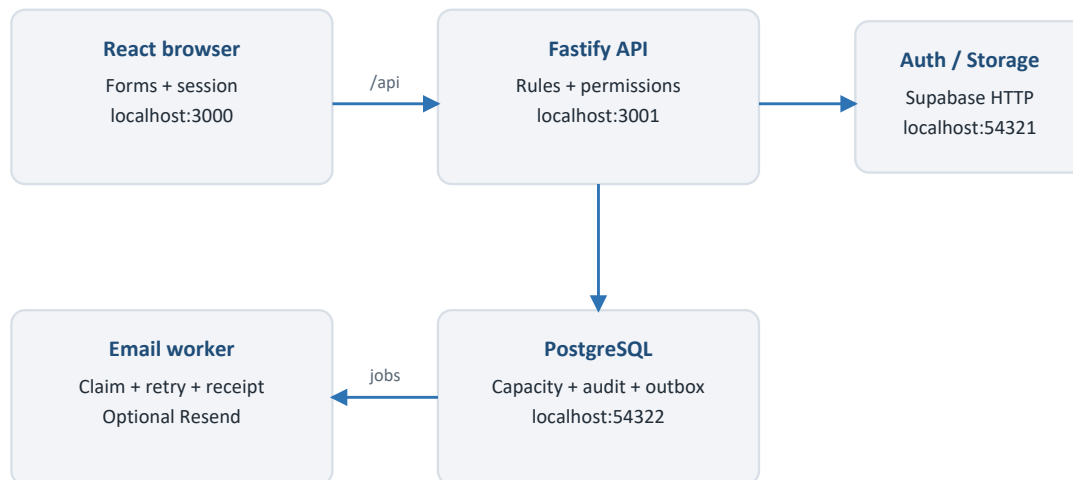
1. What you are building	3
2. Understand the windows and addresses	4
3. First setup: tools and dependencies	5
4. First setup: start Supabase	5
5. First setup: create the correct .env file	6
6. Start the website and check it	7
7. Daily startup and shutdown	7
8. Create a customer and the first administrator	8
9. Follow a menu request through the code	8
10. Follow a booking and understand the database	9
11. Accounts, storage and the worker	10
12. Read the files and basic TypeScript	11
13. Troubleshoot the error you saw	12
14. Make a small change and verify it	13
15. A learning path and further references	13

1. What you are building

This is a cafe website. A visitor can read the menu, reserve a visit, and manage that reservation. A customer can create an account. Staff can manage bookings and menu items. An administrator can give another account staff access.

Full stack means working with both the visible website and the services that store its data and enforce its rules. You can learn one part at a time. Your first goal is to run the application and make one guest booking; understanding every file comes later.

The frontend runs in the browser and shows the pages. The backend receives requests, checks them, and talks to the database. The database keeps information after you close the browser. Authentication verifies which account is signed in. These are separate responsibilities.



Browser also contacts Supabase Auth directly for account sessions.

The project uses React for the frontend, TypeScript for checked JavaScript, Fastify for the backend, and Supabase for PostgreSQL, accounts and image storage. Vite starts the frontend during development. Docker runs the local Supabase services.

An optional worker sends reservation emails. You can browse, book and cancel without running it. Account confirmation emails on local Supabase go to its local mail viewer; reservation emails use a separately configured delivery provider.

2. Understand the windows and addresses

Open this project folder in VS Code. Choose Terminal > New Terminal. Select PowerShell if VS Code offers a choice. A terminal accepts commands; the editor changes files; the browser displays the website. Do not paste TypeScript code or SQL into the terminal as if it were a command.

Run commands from the folder containing package.json:

```
Set-Location 'C:\Users\harib\Desktop\Piccolo Cafe Websitr'  
Get-Location  
Test-Path package.json
```

Get-Location should show your project folder. Test-Path should print True. A command such as `npm run dev` reads the scripts defined in this folder's package.json. Running it in a different folder can start the wrong project or fail.

When a command finishes, the prompt returns. A development server keeps running and occupies its terminal. Keep that terminal open while using the website. Open a second terminal for commands such as `npm run doctor`. Ctrl+C stops the command running in the selected terminal.

localhost and 127.0.0.1 both refer to your own computer. A port is the number after the colon; it selects a service. Use localhost:3000 for the website consistently because account redirects are configured for that address.

Address	What to open there
http://localhost:3000	The cafe website
http://127.0.0.1:3001/api/health/ready	API readiness result as JSON
http://127.0.0.1:54323	Local Supabase Studio: view tables and accounts
http://127.0.0.1:54324	Local mail viewer: account confirmation/reset messages
http://127.0.0.1:54321	Supabase API gateway; use its specific API paths
127.0.0.1:54322	PostgreSQL connection; this is not a web page

If the CLI prints a different mail or Studio address, use the address in `npm run supabase:status`. Studio and the mail viewer are development tools, not pages for your cafe customers.

3. First setup: tools and dependencies

Install Node.js 22.12 or newer; Node 24 was used to verify this project. npm comes with Node.js. Install Docker Desktop for Windows and open it. Wait for the Docker engine to be running. Installing Docker and starting Docker are separate steps.

Official installers and local development references are listed at the end of this guide. If Docker reports missing WSL or virtualization, follow Docker's Windows installation instructions and restart Windows if the installer requests it.

In the project terminal, run these commands one at a time:

```
node --version
npm --version
docker info --format '{{.ServerVersion}}'
npm ci
```

Expected results: Node prints a supported version, npm prints its version, Docker prints a server version, and npm finishes installing the project's dependencies. npm ci uses package-lock.json so you get the same dependency versions as the project. It includes the pinned Supabase CLI; you do not need to install that CLI separately.

If Docker reports that it cannot connect to the engine, stop here and open Docker Desktop. If Node or npm is not recognized, finish installing Node and open a new VS Code terminal. If PowerShell blocks npm.ps1, try npm.cmd with the same arguments instead of changing Windows-wide execution policy.

An npm vulnerability summary is a dependency report, not proof that setup failed. Do not run npm audit fix --force while learning this project: it can introduce incompatible dependency upgrades. Treat dependency updates as a separate reviewed task.

4. First setup: start Supabase

Run:

```
npm run supabase:start
```

The first run downloads Docker images and can take several minutes. Wait for it to finish successfully. It starts the database, authentication, storage, Studio and local mail tools. On a fresh local database, it also applies this project's SQL migrations and development seed.

Expected result: the command reports a started local development setup and local service addresses. The CLI may print local keys; keep that output private. Do not run supabase init: this repository already contains supabase/config.toml and migrations.

Check the running stack:

```
npm run supabase:status
```

If startup reports a port already in use, another local service may occupy a configured port. Check the named service before stopping it. Do not delete Docker volumes or run database reset to troubleshoot a startup failure; those actions can erase local records.

This setup runs on your computer. It does not deploy the website or migrate the live cafe. The development menu has sample items and prices, identified as samples in the application.

5. First setup: create the correct .env file

Run:

```
npm run setup:local  
npm run doctor
```

setup:local reads credentials from your running local Supabase instance, writes them to .env, and configures the restricted cafe_api database login. It generates long random passwords/secrets when needed. If .env already exists, it backs it up under the ignored .local-backup folder. On repeated setup, it keeps an existing valid runtime password and HMAC secrets.

The setup script refuses to overwrite a configured database or Supabase URL outside your own computer. It is intended for this local development stack. For hosted staging or production, follow docs/operations.md with the correct operator credentials.

Expected result: setup reports that .env was saved and cafe_api was configured. doctor prints PASS for configuration, the database and Supabase Auth. Only continue when those checks pass.

There are two similarly named files:

- .env.example is a shareable template that documents setting names. The backend does not load it.
- .env contains your real settings. The backend loads it from the project root, and Vite also reads it. Git ignores it because it contains passwords and server keys.

An environment variable is a named setting. In .env, DATABASE_URL identifies the database connection; SUPABASE_URL identifies the authentication/storage service. A URL is an address, and a key is a credential. Do not put terminal commands, import statements or TypeScript code in .env.

If you edit .env manually, save it as plain UTF-8 text with the exact filename .env, not .env.txt. Stop and restart npm run dev after changing settings. This project does not use env_notepad as configuration.

VITE_SUPABASE_URL and VITE_SUPABASE_PUBLISHABLE_KEY are intended for browser use. DATABASE_URL, ADMIN_DATABASE_URL, SUPABASE_SECRET_KEY, GUEST_TOKEN_SECRET and RATE_LIMIT_SECRET belong on the server. Never add a VITE_ prefix to a secret to make it work in the browser.

DATABASE_URL uses cafe_api, a restricted application role. ADMIN_DATABASE_URL uses the local database owner for one-time operator tasks. setup:local provisions the restricted login for you; you do not need to run db:role or set CAFE_API_PASSWORD by hand for this beginner setup.

6. Start the website and check it

In a terminal, run:

```
npm run dev
```

The command first checks configuration, then starts Vite and the API. Expected result: a frontend address on port 3000 and an API listening on port 3001. Leave this terminal running.

Open <http://localhost:3000> in your browser. In another tab, open <http://127.0.0.1:3001/api/health/ready>. The readiness page should return a JSON result with HTTP 200. It is an API response, so seeing data rather than a designed page is normal.

Browse the menu. You should see development sample items fetched from the database. Open the booking page and select a future date, a time and a guest count. Submit your name, email and phone. Keep the resulting private reservation link. Check the displayed details, then cancel the reservation from its management page.

To inspect a request, press F12 in the browser, choose Network, reload the menu, and select the request containing `/api/menu`. Look at its status and Response tab. A 200 response means the request succeeded. This shows that the displayed items came from the API.

`npm start` starts the frontend only. `npm run dev:api` starts the backend only. For normal development, use `npm run dev` so both run together. A visible homepage alone does not establish that bookings or accounts work.

7. Daily startup and shutdown

After the first setup, the usual routine is:

1. Open Docker Desktop and wait for its engine.
2. Open the project folder and terminal.
3. Run `npm run supabase:start` and wait for success.
4. Run `npm run doctor`.
5. Run `npm run dev` and open <http://localhost:3000>.

You do not need `npm ci` or `setup:local` every morning. Run `npm ci` when installing a fresh clone or after `package-lock.json` changes. Run `setup:local` again if the local credentials change or doctor reports a key/login mismatch.

When finished, press Ctrl+C in the development terminal. Then run this in an available terminal:

```
npm run supabase:stop
```

Normal Supabase stop preserves local data. Stop only this project's stack; unrelated Docker projects can remain running. Starting it again should retain accounts, menu edits and reservations. Database reset or deleting volumes is a different operation.

8. Create a customer and the first administrator

With the app running, open <http://localhost:3000/account>. Choose account creation, enter an email address and a password of at least 12 characters, and submit. Open the local mail viewer reported by `supabase:status`. Find the confirmation message and follow its link back to the cafe website. Sign in with that account.

The local mail viewer receives development messages even when your test address has no real mailbox. Account confirmation and password reset are handled by Supabase Auth. A guest booking does not automatically create an account or appear as an account booking later.

New accounts are customers. They cannot access staff functions. To give the first confirmed account administrator access, open local Supabase Studio, choose Authentication > Users, and copy that user's ID. This is a UUID such as a string of letters/numbers separated by hyphens; it is not the email address.

In a second project terminal, replace `YOUR_CONFIRMED_USER_UUID` below with the actual copied ID:

```
npm run admin:bootstrap -- --user-id YOUR_CONFIRMED_USER_UUID
npm run admin:bootstrap -- --user-id YOUR_CONFIRMED_USER_UUID --apply
```

The first command validates the change and rolls it back. The second applies it. The `--` before `--user-id` passes arguments through npm to the script. Bootstrap requires a confirmed user and refuses to create another first administrator once one exists.

Refresh the account page or sign out and sign in again. Open the staff dashboard at `/admin/dashboard`. Try reservation search, a valid status change and menu editing. The Team access screen is available to administrators. Use it for later role assignments.

9. Follow a menu request through the code

Reading the code is easier when you follow one user action. Begin with the menu, which is simpler than account security or reservation concurrency.

1. `src/app/main.tsx` starts React and provides routing and authentication context.
2. `src/app/App.tsx` selects the page for the current browser path.
3. `src/app/Menu.tsx` stores filter choices and asks for menu data.
4. `src/app/hooks.tsx` manages loading, errors and returned data.
5. `src/app/api.ts` sends an HTTP request using `fetch` and attaches a signed-in account token when present.
6. `vite.config.ts` forwards development requests starting with `/api` to port 3001. This forwarding is called a proxy.
7. `server/app.ts` registers the API routes and shared request/error handling.
8. `server/modules/menu.ts` validates filters, queries PostgreSQL and returns menu data as JSON.
9. React receives the result and renders the list. Loading and error feedback come from the same request state.

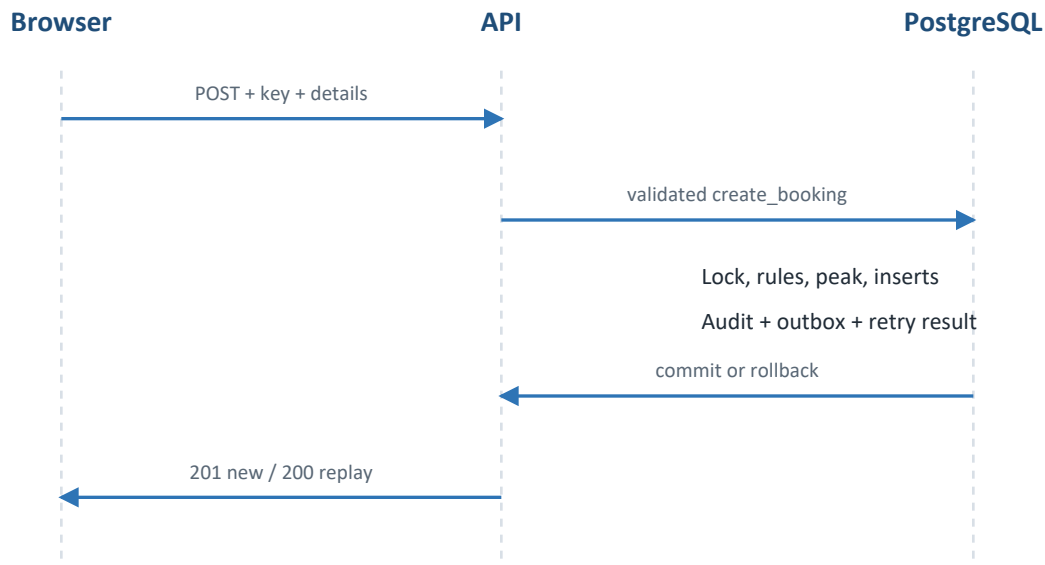
HTTP is the request/response protocol between the browser and server. GET reads data; POST creates or performs an action; PATCH changes part of a resource. A route combines a method and a path, such as GET /api/menu. JSON is text representing objects and lists that both sides can read.

For example, a simplified JSON object can look like this:

```
{
  "name": "Cappuccino",
  "price_paise": 18000,
  "published": true
}
```

The names before colons are keys, and the values after them are data. 18000 paise is 180 rupees. The actual API returns more fields and a paginated list; this small example explains the syntax.

10. Follow a booking and understand the database



Booking.tsx collects the form values and obtains available time slots. The browser sends the chosen time, guest count and contact details to the reservations API. booking-rules.ts checks the shape of the request and normalizes values. reservations.ts asks PostgreSQL to create the reservation through the database function.

The SQL migrations define tables and the booking transaction. A table stores rows of related information. A reservation row represents one visit. Columns hold values such as its ID, start time, guests, status and version. An ID uniquely identifies a row; a foreign key connects it to another row.

Table or area	What it stores
public.categories / public.menu_items	Menu groups and items
public.profiles	Account profile and customer/staff/admin role

Table or area	What it stores
public.reservations	Visit details and current status
cafe_private.settings	Capacity, hours, durations and booking rules
cafe_private.guest_access	Hashed private guest credentials
cafe_private.notification_outbox	Durable reservation email jobs
Reservation audit data	Who changed a booking and its previous/new state

Use Studio to view local rows while testing. Avoid editing reservation state directly in a table while learning: the API and database functions enforce permissions, transitions and audit behavior together.

A transaction is a group of database operations that all succeed or all roll back. The booking transaction checks capacity and stores the reservation, guest access, audit information and notification event together. A database lock coordinates simultaneous bookings, so two API instances cannot both reserve the same remaining capacity.

An idempotency key identifies one booking attempt. If the browser retries the same request after a connection problem, the server can return the original result instead of creating another booking. Reusing that key for different details is rejected. This is why booking logic has more checks than a simple INSERT.

Guest access uses a private token. Keep the management link private; another person holding it may manage that guest reservation. Account access uses a verified Supabase session and the server checks the database profile's role. Changing a button or browser data does not grant permission.

A version is an increasing number used when staff edit a booking or menu item. If another person changed the record first, an old version is rejected and the screen must reload. This prevents silently overwriting someone else's changes.

11. Accounts, storage and the worker

Account.tsx chooses signup, login, reset or password change, then calls Supabase Auth. Auth.tsx keeps the current session available to other React components. On a protected API request, server/modules/auth.ts verifies the token with Supabase and reads the account's role from PostgreSQL.

Authentication answers who is signed in. Authorization answers what that person can do. A customer can manage their own account bookings; staff can operate cafe tools; an administrator can manage team roles. The server enforces these rules as well as the visible interface.

StaffMenu.tsx edits menu items and submits image uploads through the API. The backend validates the upload and writes to Supabase Storage. PostgreSQL stores the image path; Storage stores the image bytes. A public menu image can be viewed without an account, but uploading requires staff permission.

server/worker.ts runs separately from the API. It checks stored notification jobs, claims one, sends it, and marks success or schedules a retry. A lease gives a worker temporary ownership; it expires after a crash so another worker can recover the job. A fencing token prevents an old worker from acknowledging a newly claimed job.

Reservation email delivery is optional for your first run. Later, configure `RESEND_API_KEY` and a verified `EMAIL_FROM` in `.env` using `docs/operations.md`, then open a separate terminal and run `npm run worker`. Without these credentials, the worker refuses to claim jobs. A booking can succeed while an email is still queued.

12. Read the files and basic TypeScript

Start with this reading order: `shared/types.ts`, `Menu.tsx`, `hooks.tsx`, `api.ts`, `server/index.ts`, `server/app.ts`, `server/modules/menu.ts`. Then follow `Booking.tsx`, `booking-rules.ts`, `reservations.ts` and the SQL migrations. Read `Auth` and `notifications` after those flows make sense.

File or area	Why it exists
<code>package.json</code>	Project dependencies and named npm commands
<code>package-lock.json</code>	Exact installed dependency versions
<code>.env.example</code> / <code>.env</code>	Settings template / your private configuration
<code>src/app/*.tsx</code>	React screens and UI helpers
<code>shared/types.ts</code>	Data shapes shared by frontend and API
<code>server/index.ts</code> / <code>app.ts</code>	Start the API / build and register its routes
<code>server/config.ts</code> / <code>db.ts</code>	Validate settings / connect to PostgreSQL
<code>server/validation.ts</code> / <code>services.ts</code>	Common input checks / shared API services
<code>supabase/migrations/*.sql</code>	Versioned database structure and rules
<code>scripts</code>	Setup, maintenance and verification commands
<code>tests</code>	Automated examples that verify behavior

JavaScript runs in the browser and in Node.js. TypeScript adds checks while developing; its types do not replace runtime validation of internet requests. `.tsx` files can contain JSX, the HTML-like syntax used by React.

```
interface MenuItem {
  name: string;
  price_paise: number;
}

function priceInRupees(item: MenuItem): number {
  return item.price_paise / 100;
}
```

interface describes a data shape. `item: MenuItem` means the input must match that shape. The final `number` describes the return type. The division converts paise to rupees. TypeScript can catch passing text where a number is expected before you run the app.

`const` declares a value you do not reassign; `let` is for values you do reassign. `import` uses code from another file or library; `export` lets another file use your code. `async` marks a function that completes asynchronously, and `await` waits for its result. `try/catch/finally` handles success, failure and cleanup.

`useState` stores React UI state, such as a selected filter. `useEffect` runs work when relevant values change, such as loading a new page of results. A component is a function that returns UI. Props are inputs passed into a component. The shared `Page<T>` type means a page of items, where `T` supplies the item type.

You do not need to memorize these features immediately. While reading a function, identify its inputs, its returned value and any side effect such as sending a request or writing a row.

13. Troubleshoot the error you saw

The reported errors had a chain of causes: the API could not load usable settings from `.env`; the frontend was still running; frontend requests to the missing API then reported `ECONNREFUSED` on port 3001. Also, local Supabase had not started. Starting only the frontend cannot resolve that chain.

Your local settings were placed in `.env.example`, and `.env` did not contain parseable `KEY=value` settings. The repair preserved both originals under `.local-backup`, placed settings in `.env`, and restored the shareable template. `setup:local` now obtains the actual local credentials rather than relying on manually copied key text.

If configuration fails again, check the filename and run `setup:local` after Supabase is ready. If doctor reports a database failure, check that `supabase:start` succeeded. If doctor reports an Auth key failure, rerun `setup:local`. Restart the development terminal after either repair.

The `ADMIN_DATABASE_URL` error means an operator command lacked its database connection setting. It is different from `DATABASE_URL`, the application's restricted connection. The automated local setup fills both and provisions the login.

The earlier `assertRuntimeRole` export and hot-reload messages occurred while files were changing. The current `db.ts` exports that function. Stop and restart `npm run dev` to load a consistent current version. If the import error persists after restart, use `npm run typecheck` to identify a genuine code problem.

If port 3000 is already occupied, stop your previous cafe development terminal with `Ctrl+C`. Vite can otherwise choose a different port, which does not match the configured account redirect address. Do not stop an unrelated process without checking what it is.

A Browserslist outdated-data warning is separate from missing database settings. The first actionable startup error matters most. When asking for help, include the command, the first error, and whether doctor passed. Share variable names and redacted output, never passwords, server keys or private booking links.

14. Make a small change and verify it

First edit a visible sentence in `src/app/Home.tsx` while `npm run dev` is running. Save the file and observe the browser update. This demonstrates React/Vite development without changing database rules.

Then change a menu sample through the staff menu screen and reload the public menu. This demonstrates a backend/database change. Try filtering the menu and watch the Network requests. Finally, trace one guest booking through the files listed earlier.

From a second terminal, these checks are useful after code changes:

```
npm run typecheck
npm run lint
npm test
npm run build
```

`typecheck` checks TypeScript; `lint` checks code conventions and common mistakes; `test` runs the unit tests; `build` creates frontend `build/` and compiled backend `dist/` output. Successful checks do not prove a production deployment is complete.

For database verification, `npm run test:embedded` creates an isolated test database. Before `npm run test:e2e`, stop the normal dev terminal to free ports 3001/3000. Its first run needs `npx playwright install chromium`. Keep test databases separate from development/production data.

`npm run test:security` verifies actual local Supabase account/Data API/Storage restrictions and creates/removes its own test records. Run it after Supabase and `setup:local` succeed. See `docs/verification.md` for evidence and remaining acceptance checks.

15. A learning path and further references

Practise these milestones: run the website; explain a menu request; make and cancel a guest booking; confirm an account; use staff tools; explain transactions and retries. Repeat each until you can describe what happens.

Learn JavaScript objects, arrays, functions and promises; React components, props, state and effects; HTTP methods, status codes and JSON; SQL SELECT, INSERT, UPDATE and transactions; and Git edits, commits, pushes, branches and merges.

This upgrade is on `upgrade/supabase-reliable-reservations`. An old GitHub commit link always shows its old snapshot. A branch push, merge into main and deployment are separate steps. Read the system handbook and `docs/system-design.md` for advanced reliability choices after the basics.

Official references checked for this guide:

- [Node.js downloads](#): install Node and npm.
- [Docker Desktop for Windows](#): Windows requirements, installation and WSL setup.
- [Supabase local development CLI](#): Docker-based local services, first downloads, credentials and normal stop.
- [Vite environment variables](#): `.env` loading and why browser-exposed `VITE_` values must be public.